

AUDITORIA INTEGRAL

CREARD - Reserva de Canchas Deportivas

Aplicacion: <https://creard.vercel.app>

Fecha: 31 de Agosto de 2026

Tipo: Auditoria No Destructiva

Alcance: Funcional, Seguridad, UX, Base de Datos, Pagos

1. RESUMEN EJECUTIVO	3
2. TOP 10 PROBLEMAS MAS IMPORTANTES	4
3. HALLAZGOS DETALLADOS	5
3.1 Vulnerabilidades de Autenticacion y Autorizacion	5
3.2 Vulnerabilidades del Sistema de Pagos	7
3.3 Concurrencia y Reservas Simultaneas	10
3.4 Problemas de Zona Horaria	11
3.5 Problemas de UX/UI	11
4. AUDITORIA POR MODULO	13
5. PROBLEMAS QUE PUEDEN GENERAR PERDIDA ECONOMICA	14
6. PROBLEMAS DE SEGURIDAD	15
6.1 Criticos	15
6.2 Altos	15
6.3 Medios	15
6.4 Bajos	15
7. PROBLEMAS DE RESERVAS	17
8. PROBLEMAS DEL PANEL ADMINISTRATIVO	18
9. PROBLEMAS DEL USUARIO FINAL	19
10. PLAN DE CORRECCION	20
FASE 1 - URGENTE (P0)	20
FASE 2 - ALTA PRIORIDAD (P1)	20
FASE 3 - MEJORAS (P2)	20
FASE 4 - OPTIMIZACION (P3)	21
11. CHECKLIST FINAL DE PRODUCCION	22

1. RESUMEN EJECUTIVO

La aplicacion CREARD es un sistema de reserva y gestion de canchas deportivas de futbol y voleibol desarrollado en Next.js 16 con Firebase/Firestore como backend. La aplicacion presenta una arquitectura funcional con un flujo de reserva completo que incluye seleccion de deporte, cancha, fecha y horario, pero presenta vulnerabilidades criticas de seguridad y problemas funcionales que deben resolverse antes de su puesta en produccion.

Se identificaron un total de **52 hallazgos** distribuidos en 4 niveles de prioridad. La aplicacion cuenta con un sistema de autenticacion basado en Firebase Auth con verificacion de tokens y roles almacenados en Firestore, lo cual es una base solida. Sin embargo, multiples endpoints de la API carecen de autenticacion adecuada, el sistema de pagos tiene fallas criticas de validacion, y existen credenciales administrativas expuestas en el codigo fuente. Ademas, problemas de concurrencia en la edicion de reservas y la ausencia de idempotencia en webhooks de Culqi representan riesgos financieros reales.

Indicador	Valor
Nivel de Riesgo	ALTO
Funcionalidades Correctas (estimado)	55%
Problemas Criticos (P0)	14
Problemas Altos (P1)	12
Problemas Medios (P2)	16
Problemas Bajos (P3)	10
Lista para Produccion	NO - CONDICIONAL

Principales Riesgos: (1) Múltiples endpoints sin autenticación exponen datos financieros y permiten escritura no autorizada. (2) El API de pagos acepta montos y estados desde el cliente sin validar contra el servidor. (3) Credenciales administrativas hardcodeadas en el código fuente. (4) Ausencia de idempotencia en webhooks de Culqi permite doble conteo de pagos. (5) Endpoints de configuración one-time (setup) permanecen desplegados con secretos débiles.

2. TOP 10 PROBLEMAS MAS IMPORTANTES

Prioridad	Problema	Impacto	Riesgo	Solucion
P0	Endpoints sin autenticacion (stats, expenses, clients, settings)	Datos financieros publicos, escritura no autorizada	Critico	Agregar requireAuth a todos los endpoints
P0	Credenciales admin hardcodeadas en codigo fuente	Acceso no autorizado al sistema completo	Critico	Eliminar hardcodes, usar solo env vars
P0	API /api/payments acepta monto y estado desde el cliente	Manipulacion de pagos, fraude	Critico	Validar monto contra booking.remaining_amount en servidor
P0	Webhook Culqi sin idempotencia	Doble conteo de pagos	Critico	Dedicar por charge_id, verificar antes de procesar
P0	Setup endpoints desplegados con secretos debiles	Cualquiera puede crear super_admin	Critico	Eliminar /api/setup/* del despliegue
P1	Precios de equipamiento desde el cliente sin validar	Equipamiento gratis o a precio manipulado	Alto	Validar unit_price contra coleccion equipment
P1	Edicion de reservas (PUT) sin transaccion	Reservas duplicadas con ediciones concurrentes	Alto	Envolver en db.runTransaction()
P1	createPayment escribe en 2 colecciones sin transaccion	Registros de pago huérfanos	Alto	Usar batch write atomico
P1	Falta de ownership check en payment-validation POST	Usuario puede marcar reservas ajenas	Alto	Verificar booking.user_id === authUser.id
P1	Problemas de zona horaria en frontend	Fechas/horas incorrectas para usuarios fuera de Lima	Alto	Usar America/Lima en todas las comparaciones

3. HALLAZGOS DETALLADOS

3.1 Vulnerabilidades de Autenticacion y Autorizacion

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[PO CRITICO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[PO CRITICO]', textColor=Color(.8,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [FAL-001] Endpoints completamente sin autenticacion

Modulo: Seguridad

Problema: Los endpoints /api/stats, /api/expenses (GET/POST/DELETE) y /api/clients no tienen ninguna verificacion de autenticacion. Cualquier usuario no autenticado puede acceder a datos financieros completos, crear gastos falsos, eliminar gastos reales, y enumerar datos personales de todos los usuarios incluyendo nombres, correos y telefonos.

Como reproducirlo: Acceder directamente a /api/stats o /api/clients sin token de autenticacion.

Resultado actual: Se retornan datos financieros completos y datos PII de usuarios sin restriccion.

Resultado esperado: Deberia retornar 401/403 para usuarios no autenticados.

Impacto: Exposicion completa de datos financieros y personales. Posible cumplimiento ilegal con regulaciones de proteccion de datos.

Riesgo: Critico | **Causa:** Los endpoints no llaman a requireAuth() ni requireAnyAuth(). La proteccion se basa unicamente en la suposicion de que el cliente no conoce las URLs.

Solucion: Agregar requireAuth(request, "admin") al inicio de cada handler. Para /api/clients, requerir admin para listar todos, y requireAnyAuth para busqueda propia.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[PO CRITICO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[PO CRITICO]', textColor=Color(.8,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [FAL-002] Credenciales administrativas hardcodeadas

Modulo: Seguridad

Problema: En /api/auth/route.ts lineas 120-121 y /api/setup/create-admin/route.ts lineas 37-38, se encuentran hardcodeadas las credenciales del administrador: email weilhelmn@gmail.com y contrasena Creard2025!. Ademas, /api/setup/create-admin retorna la contrasena en el cuerpo de la respuesta HTTP (linea 106).

Como reproducirlo: Buscar las cadenas "Creard2025!" o "weilhelmn@gmail.com" en el codigo fuente del repositorio de GitHub.

Resultado actual: Cualquier persona con acceso al repositorio obtiene acceso completo como super_administrador.

Resultado esperado: Las credenciales deben estar exclusivamente en variables de entorno del servidor.

Impacto: Acceso no autorizado al sistema completo con permisos de super_administrador.

Riesgo: Critico | **Causa:** El patron de demo mode hardcodea credenciales directamente en el codigo. El endpoint setup retorna la contrasena en JSON.

Solucion: Eliminar todo hardcode. Usar solo FIREBASE_SERVICE_ACCOUNT_KEY y variables de entorno. Eliminar el endpoint /api/setup/create-admin del despliegue de produccion.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P0 CRITICO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P0 CRITICO]', textColor=Color(.8,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [FAL-003] Secreto de setup debil y hardcodeado

Modulo: Seguridad

Problema: El endpoint /api/setup/create-admin utiliza process.env.SETUP_SECRET con valor por defecto "creard-setup-2025" (linea 13). Si la variable de entorno no esta configurada, cualquiera que conozca el valor por defecto puede crear cuentas de super_administrador.

Como reproducirlo: Enviar POST a /api/setup/create-admin con el header Authorization conteniendo el valor por defecto.

Resultado actual: Se crea un nuevo super_admin o se resetea la contraseña del existente.

Resultado esperado: El endpoint deberia no existir en produccion.

Impacto: Escalada de privilegios a super_admin.

Riesgo: Critico | **Causa:** Valor por defecto predecible. Endpoint deberia eliminarse antes de produccion.

Solucion: Eliminar completamente /api/setup/create-admin y /api/setup/fix-names del codigo desplegado.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P1 ALTO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P1 ALTO]', textColor=Color(.901961,.45098,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [FAL-004] Auth rota en /api/admin/settings GET descartada

Modulo: Seguridad

Problema: En /api/admin/settings/route.ts linea 39, el resultado de requireAuth() se captura con .catch(() => null) y se ignora completamente. La configuracion del sitio se retorna sin importar si el usuario esta autenticado o no.

Como reproducirlo: Llamar a GET /api/admin/settings sin token de autenticacion.

Resultado actual: Se retorna la configuracion completa del sitio incluyendo datos sensibles.

Resultado esperado: Deberia retornar 401.

Impacto: Fuga de informacion de configuracion del sitio.

Riesgo: Alto | **Causa:** El resultado de requireAuth se descarta con .catch(() => null).

Solucion: Verificar el resultado de requireAuth y retornar error si es instancia de NextResponse.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P1 ALTO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P1 ALTO]', textColor=Color(.901961,.45098,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [FAL-005] Endpoints admin sin requerir rol admin

Modulo: Seguridad

Problema: Los endpoints /api/admin/gallery (POST/PUT/DELETE), /api/admin/news (POST/PUT/DELETE) y /api/admin/courts (PUT) utilizan requireAuth(request) sin el parametro de rol requerido. Cualquier usuario autenticado con rol "user" puede crear, modificar o eliminar galeria, noticias y precios de canchas.

Como reproducirlo: Un usuario regular envia PUT a /api/admin/courts con un precio modificado.

Resultado actual: El precio de la cancha se actualiza correctamente.

Resultado esperado: Deberia retornar 403 Forbidden.

Impacto: Cualquier usuario puede modificar precios, contenido y galeria del sitio.

Riesgo: Alto | **Causa:** requireAuth(request) sin segundo parametro acepta cualquier rol autenticado.

Solucion: Cambiar a requireAuth(request, "admin") en todos los endpoints de escritura bajo /api/admin/.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P1 ALTO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P1 ALTO]', textColor=Color(.901961,.45098,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [FAL-006] SSRF en /api/notifications/dispatch

Modulo: Seguridad

Problema: El endpoint acepta URLs de webhook (Google Chat, WhatsApp) desde el cuerpo de la solicitud y las utiliza directamente en fetch() sin validacion. Un administrador (o un atacante que comprometa una cuenta) podria apuntar estas URLs a servicios internos como http://169.254.169.254/ (metadatos de cloud) o http://localhost:8080.

Como reproducirlo: Enviar POST a /api/notifications/dispatch con webhookUrl apuntando a un servicio interno.

Resultado actual: El servidor realiza una solicitud al servicio interno.

Resultado esperado: Deberia validarse la URL contra una lista blanca de dominios.

Impacto: Acceso a servicios internos, fuga de informacion de metadatos de cloud.

Riesgo: Alto | **Causa:** No hay validacion de URL. fetch() acepta cualquier URL incluyendo direcciones IP privadas.

Solucion: Implementar validacion de URL: bloquear IPs privadas (10.x, 172.16-31.x, 192.168.x, 169.254.x) y restringir a dominios permitidos.

3.2 Vulnerabilidades del Sistema de Pagos

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[PO CRITICO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[PO CRITICO]', textColor=Color(.8,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [PAG-001] API payments acepta monto y estado desde el cliente sin validacion

Modulo: Pagos

Problema: El endpoint /api/payments (POST, linea 115) toma el monto directamente del cuerpo de la solicitud con parseFloat(amount) sin validar contra el monto esperado segun la reserva. Ademas, el estado del pago (linea 118) acepta cualquier valor incluyendo "completed" sin verificar que el pago fue realmente realizado.

Como reproducirlo: Enviar POST a /api/payments con { bookingId: "xxx", amount: 0.01, status: "completed" } para una reserva de S/100.

Resultado actual: Se crea un registro de pago por S/0.01 con estado "completed" y la reserva se actualiza.

Resultado esperado: El servidor deberia calcular el monto esperado a partir de la reserva y rechazar montos que no coincidan.

Impacto: Un usuario puede registrar pagos por montos arbitrarios, marcar pagos como completados sin pagar, y manipular el estado financiero de las reservas.

Riesgo: Critico | **Causa:** Ausencia total de validacion server-side del monto. El campo status se acepta sin cuestionar.

Solucion: Antes de crear el pago, obtener la reserva con getBookingById(). Calcular el monto esperado (remaining_amount o total_price segun type). Rechazar si amount difiere del esperado en mas de S/0.50. Ignorar el campo status del cliente y usar "pending" por defecto.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[PO CRITICO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[PO CRITICO]', textColor=Color(.8,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [PAG-002] Webhook de Culqi sin idempotencia - doble conteo de pagos

Modulo: Pagos

Problema: El handler de webhook (webhooks/culqi/route.ts lineas 131-145) actualiza el monto pagado sin verificar si el webhook ya fue procesado anteriormente. Culqi garantiza entrega "at least once", por lo que duplicados son esperados. Un webhook duplicado anadira el monto dos veces al advance_amount de la reserva.

Como reproducirlo: Simular un reenvio de webhook de Culqi con el mismo charge_id.

Resultado actual: El advance_amount se incrementa dos veces, sobrepagando la reserva.

Resultado esperado: Cada webhook deberia verificarse por charge_id. Si ya fue procesado, retornar 200 sin modificar nada.

Impacto: El monto pagado se duplica, causando inconsistencias financieras y reportes incorrectos.

Riesgo: Critico | **Causa:** No existe mecanismo de deduplicacion. Culqi puede reenviar webhooks.

Solucion: Antes de procesar, consultar si ya existe un registro de pago con ese external_ref/charge_id. Si existe, retornar 200 OK inmediatamente. Usar el charge_id como clave de idempotencia.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P1 ALTO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P1 ALTO]', textColor=Color(.901961,.45098,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [PAG-003] Precios de equipamiento desde el cliente sin validacion server-side

Modulo: Pagos

Problema: En /api/bookings/route.ts lineas 518-522, los precios unitarios de los items de equipamiento (eq.unit_price) se toman directamente del cuerpo de la solicitud sin verificar contra la coleccion "equipment" en Firestore. Un usuario podria enviar unit_price: 0.01 para obtener equipamiento gratis.

Como reproducirlo: Enviar una reserva con equipmentItems: [{ equipment_id: "xxx", quantity: 5, unit_price: 0.01 }].

Resultado actual: El subtotal de equipamiento se calcula con el precio manipulado.

Resultado esperado: El servidor deberia buscar cada equipment_id en la coleccion equipment y usar el precio real.

Impacto: Perdida economica por equipamiento a precio manipulado.

Riesgo: Alto | **Causa:** El precio del servidor se recalcula para canchas pero no para equipamiento.

Solucion: En el handler POST de /api/bookings, antes de calcular el subtotal, hacer getCourtById o getEquipment para obtener el precio real de cada item.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P1 ALTO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P1 ALTO]', textColor=Color(.901961,.45098,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [PAG-004] Cargo expirado de Culqi no libera la cancha

Modulo: Pagos

Problema: Cuando Culqi envia un webhook de cargo expirado (handleExpiredCharge, lineas 209-223), el pago se marca como "expired" pero la reserva permanece con status "reserved" y el horario permanece bloqueado. La cancha queda ocupada para siempre hasta que un admin interviene manualmente o el cleanup lazy la libere.

Como reproducirlo: Crear una reserva con pago Culqi, no completar el pago, esperar a que expire.

Resultado actual: La reserva permanece "reserved" y la cancha aparece ocupada.

Resultado esperado: El handler deberia cancelar la reserva y liberar el slot.

Impacto: Canchas bloqueadas indefinidamente por pagos expirados.

Riesgo: Alto | **Causa:** El handler solo actualiza el estado del pago, no el de la reserva.

Solucion: En handleExpiredCharge, agregar logica para cancelar la reserva (status: "cancelled", slot_status: "available") cuando se trate de un pago de adelanto.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P1 ALTO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P1 ALTO]', textColor=Color(.901961,.45098,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [PAG-005] createPayment escribe en 2 colecciones sin transaccion

Modulo: Pagos

Problema: La funcion createPayment (db.ts lineas 567-578) escribe el pago tanto en la subcoleccion bookings/{id}/payments como en la coleccion de nivel superior payments. Estas son dos operaciones de escritura separadas sin transaccion. Si la segunda falla, se crea un registro huérfano en la subcoleccion que no aparece en los reportes administrativos.

Como reproducirlo: Simular un fallo de red entre la primera y segunda escritura.

Resultado actual: El pago existe en la subcoleccion pero no en la coleccion principal.

Resultado esperado: Ambas escrituras deberian ser atómicas usando db.batch().

Impacto: Pagos que no aparecen en el panel de administracion, inconsistencia de datos.

Riesgo: Alto | **Causa:** No hay garantia de atomicidad entre las dos escrituras.

Solucion: Reemplazar las dos escrituras individuales con db.batch().add() para garantizar atomicidad.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P2 MEDIO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P2 MEDIO]', textColor=Color(.8,.6,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [PAG-006] Falta de ownership check en payment-validation POST

Modulo: Pagos

Problema: El endpoint POST /api/payment-validation (linea 62) usa requireAuth(request, "user") permitiendo a cualquier usuario autenticado marcar reservas como pendientes de validacion. No verifica que las reservas pertenecen al usuario autenticado. Un usuario podria marcar reservas de otros como pendientes.

Como reproducirlo: Enviar POST a /api/payment-validation con bookingIds que pertenecen a otro usuario.

Resultado actual: Las reservas de otro usuario se marcan como payment_pending.

Resultado esperado: Verificar booking.user_id === authUser.id para cada bookingId.

Impacto: Un usuario podria interferir con reservas de otros.

Riesgo: Medio | **Causa:** Solo se verifica autenticacion, no autorizacion de propiedad.

Solucion: Antes de actualizar, obtener cada reserva y verificar que user_id coincide con el usuario autenticado.

3.3 Concurrencia y Reservas Simultaneas

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P2 MEDIO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P2 MEDIO]', textColor=Color(.8,.6,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [CON-001] Creacion de reserva USA transaccion (correcto) pero edicion NO

Problema: La creacion de reservas (POST /api/bookings) usa correctamente db.runTransaction() para verificar disponibilidad y crear la reserva atomicamente (lineas 565-621). Sin embargo, las operaciones PUT para extender tiempo (lineas 790-848) y editar reserva (lineas 944-968) verifican conflictos con getBookings() y luego actualizan individualmente con updateBooking(), creando una condicion de carrera TOCTOU.

Impacto: Reservas solapadas en escenarios de edicion concurrente.

Solucion: Envolver la logica de verificacion + actualizacion de PUT en db.runTransaction().

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[P2 MEDIO]' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7, greek=0, italic=0, link=[], rise=0, text='[P2 MEDIO]', textColor=Color(.8,.6,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph [CON-002] Expiracion de reservas solo se ejecuta cuando alguien consulta

Problema: Las reservas con fecha de expiracion (expires_at) solo se limpian cuando un usuario llama a GET /api/bookings (lineas 397-420). No existe un cron job o funcion programada que las limpie activamente. Una reserva expirada puede permanecer visible como "reserved" hasta que alguien consulte el endpoint.

Impacto: Canchas bloqueadas por reservas expiradas que no se liberan.

Solucion: Implementar un cron job (Vercel Cron o Cloud Function) que ejecute la limpieza cada 5 minutos.

3.4 Problemas de Zona Horaria

[TZ-001] P0 CRITICO - getNextFullHour() y comparaciones de fecha usan hora local del navegador, no America/Lima. En UnifiedBookingView.tsx lineas 85-89, la funcion getNextFullHour() usa new Date() sin especificar zona horaria. Si un usuario esta en una zona horaria diferente a Lima (UTC-5), la "hora actual" sera incorrecta, permitiendo reservar horarios pasados o bloqueando horarios disponibles. Formatear fechas con toLocaleDateString sin timeZone: America/Lima produce fechas incorrectas para usuarios fuera de Peru. Solucion: Usar siempre timeZone: America/Lima en todas las llamadas a toLocaleString, toLocaleDateString y toLocaleTimeString. Para comparaciones de "ahora", usar la libreria de temporalidad o formatear explicitamente en Lima.

3.5 Problemas de UX/UI

[UX-001] P1 ALTO - BookingForm.tsx es codigo muerto con selector de pago no funcional. El componente BookingForm.tsx (921 lineas) parece estar reemplazado por UnifiedBookingView.tsx. El selector de metodos de pago (lineas 55-57, 612-650) renderiza elementos div no interactivos. Los usuarios ven opciones de pago que no responden a clics. Solucion: Eliminar BookingForm.tsx si esta obsoleto, o integrar sus funcionalidades faltantes en UnifiedBookingView.

[UX-002] P1 ALTO - Boton "Atras" desde paso de pago crea reservas huérfanas. En UnifiedBookingView.tsx, cuando el usuario esta en formStep="payment" y retrocede, la reserva ya fue creada en la base de datos. El usuario puede cambiar selecciones y crear otra reserva, dejando la primera como registro huérfano con status "reserved" que nunca sera pagado. Solucion: Cancelar la reserva existente al retroceder, o implementar el flujo de pago antes de la creacion de la reserva.

[UX-003] P2 MEDIO - Voucher de pago se imprime en blanco. El componente PaymentVoucher.tsx abre una nueva ventana con document.write() e imprime antes de que las fuentes de Google Fonts terminen de cargar. El onload se dispara antes de que las fuentes esten disponibles, resultando en texto invisible en la captura de impresion. Ademas, payment_id esta vacio para pagos Yape QR. Solucion: Usar

document.fonts.ready antes de imprimir. Para payment_id vacio, mostrar "Pendiente de asignacion" o usar el booking_code como fallback.

[UX-004] P2 MEDIO - Consistencia de estados de reserva entre vistas. BookingsView.tsx usa estados como "reserved", "payment_pending", "completed", "cancelled". ProfileView.tsx usa estados completamente diferentes: "pending", "confirmed", "partially_paid", "fully_paid". Una reserva con status "payment_pending" se mostrara con etiqueta incorrecta en el perfil del usuario. Solucion: Unificar el mapeo de estados en un archivo compartido y usarlo en todos los componentes.

[UX-005] P2 MEDIO - FeaturedCourts muestra horas disponibles aleatorias. FeaturedCourts.tsx linea 119 genera disponibilidad con $\text{Math.floor}(\text{Math.random()} * 6) + 5$ en cada render, mostrando datos completamente falsos. Solucion: Obtener disponibilidad real de la API o eliminar el dato ficticio.

[UX-006] P2 MEDIO - Inconsistencia en formato de precio. Algunas vistas usan "S/ " mientras otras usan "S/. ". Solucion: Crear un helper compartido fmtMoney() y usarlo en toda la aplicacion.

[UX-007] P1 ALTO - Barra de accion fija oculta detras de BottomNavBar en movil. Las barras inferiores fijadas en UnifiedBookingView no tienen en cuenta la altura del BottomNavBar (80px). Ambas tienen z-40 pero la barra de accion queda detras de la navegacion inferior. Solucion: Agregar bottom-20 (80px) a las barras fijadas para compensar el BottomNavBar.

[UX-008] P3 BAJO - Selector de rango de horarios no documentado. El comportamiento de seleccion por rango en handleSlotToggle (hacer clic en dos horarios separados para seleccionar todo el rango intermedio) no esta documentado ni indicado visualmente. Los usuarios no saben que esta funcionalidad existe.

4. AUDITORIA POR MODULO

Modulo	Estado	Detalle
Autenticacion	MEJORABLE	Funciona con Firebase pero tiene fallback inseguro, hardcodes y sin rate limiting
Usuarios	MEJORABLE	CRUD protegido correctamente. Setup endpoint debe eliminarse
Reservas	MEJORABLE	Creacion con transaccion (OK). Edicion sin transaccion. Expiracion lazy
Canchas	MEJORABLE	CRUD funciona. PUT sin requerir rol admin en endpoint courts
Horarios	ERROR	Problemas de zona horaria. getNextFullHour usa hora local del navegador
Pagos	CRITICO	Sin validacion de monto en servidor. Sin idempotencia en webhooks. Status desde cliente
Administracion	CRITICO	6 endpoints sin autenticacion. 3 con auth rota. 1 con SSRF
Firebase/DB	MEJORABLE	Sin reglas de seguridad desplegadas para varias colecciones. Dual write sin transaccion
Seguridad	CRITICO	Credenciales hardcodeadas. Endpoints expuestos. Setup con secreto debil
UX/UI	MEJORABLE	Buena estetica. Voucher roto. Estados inconsistentes. Formulario muerto
Responsive	MEJORABLE	Buena base. Barra de accion oculta en movil. Touch targets pequenos
Rendimiento	MEJORABLE	Sin paginacion server-side. Consultas sin limite. findPaymentByExternalRef O(N*M)
Reportes	MEJORABLE	Stats basicos funcionan. Faltan reportes detallados por cancha/deporte/periodo
Notificaciones	ERROR	Sin persistencia. Sin email/SMS. Sin recordatorios. Solo toasts en memoria

5. PROBLEMAS QUE PUEDEN GENERAR PERDIDA ECONOMICA

ID	Problema	Prioridad	Impacto Economico
PAG-001	Monto de pago aceptado desde el cliente sin validacion	P0	Un usuario puede registrar pagos por montos menores al real
PAG-002	Doble conteo de pagos por webhook Culqi duplicado	P0	Sobrepago y reportes financieros incorrectos
PAG-003	Precios de equipamiento manipulables	P1	Equipamiento gratuito o a precio reducido
PAG-004	Canchas bloqueadas por pagos Culqi expirados	P1	Ingresos perdidos por canchas no disponibles
FAL-005	Cualquier usuario puede modificar precios de canchas	P1	Precios alterados desde la interfaz de usuario
CON-002	Reservas expiradas no liberan canchas	P2	Bloqueo de canchas que podrian generar ingresos
UX-002	Reservas huérfanas por retroceso en flujo de pago	P2	Canchas bloqueadas por reservas que nunca se pagaran
FAL-004	Auth rota permite modificar configuracion del sitio	P1	Posible modificacion de datos de contacto y terminos legales

6. PROBLEMAS DE SEGURIDAD

6.1 Criticos

ID	Vulnerabilidad	Impacto	Solucion
FAL-001	Endpoints sin autenticacion (stats, expenses, clients, settings)	Datos financieros y PII completamente expuestos	Agregar requireAuth a todos
FAL-002	Credenciales admin hardcodeadas (weilhelmn@gmail.com / Creard2025!)	Acceso no autorizado como super_admin	Eliminar hardcodes del codigo
FAL-003	Setup endpoint con secreto por defecto debil	Cualquiera puede crear super_admin	Eliminar endpoint del despliegue
PAG-001	API payments acepta monto y estado del cliente	Fraude de pagos	Validar server-side contra booking
PAG-002	Webhook Culqi sin idempotencia	Doble conteo de pagos	Deduplicar por charge_id

6.2 Altos

ID	Vulnerabilidad	Impacto	Solucion
FAL-004	Auth rota en admin/settings GET	Fuga de configuracion	Verificar resultado de requireAuth
FAL-005	Admin endpoints sin requerir rol (gallery, news, courts PUT)	Escalada de privilegios	Agregar "admin" como segundo param
FAL-006	SSRF en notifications/dispatch	Acceso a servicios internos	Validar URLs, bloquear IPs privadas
PAG-003	Precios equipo sin validacion	Perdida economica	Validar contra coleccion equipment
PAG-004	Cargo expirado no libera cancha	Canchas bloqueadas	Cancelar reserva en handler
PAG-005	Dual write sin transaccion	Registros huérfanos	Usar db.batch()
TZ-001	Zona horaria inconsistente	Reservas en fecha incorrecta	Usar America/Lima en todo

6.3 Medios

ID	Vulnerabilidad	Impacto	Solucion
UX-003	XSS en voucher de pago via HTML sin sanitizar	Ejecucion de scripts en print	Sanitizar datos antes de interpolar en HTML
FAL-007	Token WhatsApp en respuesta de API	Fuga de token de autenticacion	No retornar tokens en respuestas GET
CON-001	Edicion de reservas sin transaccion	Reservas solapadas	Usar db.runTransaction() en PUT

6.4 Bajos

ID	Vulnerabilidad	Impacto	Solucion
FBK-001	Auth fallback a headers x-user-id si Firebase falla	Posible suplantacion de identidad	Retornar 401 en lugar de fallback
FBK-002	Role almacenado en localStorage	UI admin visible sin permisos	Verificar rol con llamada al servidor
FBK-003	Demo mode acepta cualquier contraseña	Solo en modo demo, sin Firebase	Documentar como riesgo aceptado de demo

7. PROBLEMAS DE RESERVAS

Aspecto	Estado	Riesgo	Solucion
Doble reserva (creacion)	Protegido con transaccion Firestore	CORRECTO	-
Doble reserva (edicion)	Edicion PUT sin transaccion - condicion de carrera	ALTO RIESGO	Envolver en runTransaction
Reservas vencidas	Solo se limpian en GET lazy	MEDIO	Implementar cron job
Horarios incorrectos (timezone)	getNextFullHour usa hora local del navegador	CRITICO	Forzar America/Lima
Fechas incorrectas	Date parsing local sin zona horaria	ALTO	Usar timeZone en todos los formatos
Reservas huérfanas	Boton atras en pago deja reserva sin pagar	MEDIO	Cancelar reserva al retroceder
Cancelaciones	Funcionan correctamente, slot liberado	CORRECTO	-
Disponibilidad	Correcta dentro de la transaccion	CORRECTO	-
Validacion de campos	Falta validacion de formato de fecha y hora	MEDIO	Validar formato YYYY-MM-DD y HH:mm
Cancha inexistente	No se verifica que courtIds existan	MEDIO	Verificar con getCourtById

8. PROBLEMAS DEL PANEL ADMINISTRATIVO

Funcion	Estado	Detalle
Gestion Usuarios	Funciona correctamente	Correcto
Gestion Canchas	PUT sin requerir rol admin	Parcial
Gestion Reservas	Edicion sin transaccion	Parcial
Validacion Pagos	Recien mejorado, funciona	Correcto
Historial Pagos	Con auditoria y filtros	Correcto
Estadisticas/Stats	Sin autenticacion - datos publicos	Error
Gastos/Expenses	Sin autenticacion en ningun metodo	Error
Galeria	POST/PUT/DELETE sin rol requerido	Parcial
Noticias	POST/PUT/DELETE sin rol requerido	Parcial
Configuracion Sitio	GET auth descartada, PUT sin rol	Error
Notificaciones	SSRF en dispatch, token en respuesta	Parcial
Adelantos Retenidos	Auth rota (authUser.role vs authUser.user.role)	Error
Reportes	Solo stats basicos, faltan reportes detallados	Mejorable

9. PROBLEMAS DEL USUARIO FINAL

El usuario final experimenta las siguientes dificultades a lo largo del flujo completo de la aplicacion:

Paso	Observacion
Registro	Sin confirmacion por email. Usuarios creados con status "pending" sin feedback claro.
Inicio Sesion	Funciona correctamente con Firebase. Demo mode acepta cualquier contrasena.
Seleccion Deporte	Funciona correctamente. Buena presentacion visual.
Seleccion Cancha	Todas muestran "Disponible" estaticamente, no refleja disponibilidad real del dia.
Seleccion Fecha	Calendario funcional. Scroll horizontal sin indicador visual.
Seleccion Horario	Seleccion por rango no documentada. Horarios pasados no bloqueados si zona horaria incorrecta.
Resumen/Precio	Precio recalculado server-side (seguro). Formato inconsistente S/ vs \$/.
Pago	Metodo de pago en BookingForm no funcional. Yape sin comprobante. Voucher se imprime en blanco.
Confirmacion	Solo toast notification. Sin email de confirmacion. Sin comprobante descargable.
Consulta Reserva	Estados inconsistentes entre BookingsView y ProfileView.
Historial Pagos	Funciona correctamente con el panel recién mejorado.
Perfil	Opciones "Notificaciones" y "Ayuda" no funcionales.

10. PLAN DE CORRECCION

FASE 1 - URGENTE (P0)

ID	Problema	Dependenc ia	Dificult ad	Prioridad	Resultado
FAL-001	Agregar auth a stats, expenses, clients, settings PUT	Ninguna	Baja	Inmediata	Endpoints protegidos
FAL-002	Eliminar credenciales hardcodeadas del codigo	Ninguna	Baja	Inmediata	Sin credenciales en codigo
FAL-003	Eliminar /api/setup/* del despliegue	Ninguna	Baja	Inmediata	Sin endpoints de setup
PAG-001	Validar monto server-side en /api/payments	db.ts	Media	Inmediata	Pagos validados
PAG-002	Agregar idempotencia a webhook Culqi	Ninguna	Media	Inmediata	Sin doble conteo

FASE 2 - ALTA PRIORIDAD (P1)

ID	Problema	Dependenc ia	Dificult ad	Prioridad	Resultado
FAL-004	Corregir auth en admin/settings GET	Ninguna	Baja	Alta	Auth verificado
FAL-005	Agregar rol a endpoints admin	Ninguna	Baja	Alta	Solo admin puede escribir
FAL-006	Validar URLs en notifications/dispatch	Ninguna	Media	Alta	Sin SSRF
PAG-003	Validar precios equipo server-side	db.ts	Media	Alta	Precios correctos
PAG-004	Liberar cancha en cargo expirado Culqi	Ninguna	Baja	Alta	Slots liberados
PAG-005	Usar batch write en createPayment	db.ts	Baja	Alta	Escritura atomica
PAG-007	Agregar ownership check en payment-validation POST	Ninguna	Baja	Alta	Solo propio usuario
TZ-001	Forzar America/Lima en todo el frontend	Ninguna	Media	Alta	Fechas correctas
UX-002	Cancelar reserva al retroceder en pago	Ninguna	Media	Alta	Sin huérfanas

FASE 3 - MEJORAS (P2)

ID	Problema	Dependenc ia	Dificult ad	Prioridad	Resultado
CON-001	Envolver PUT en transaccion	db.ts	Media	Media	Sin condiciones de carrera
CON-002	Implementar cron de limpieza	Vercel Cron	Media	Media	Reservas expiradas limpiadas

ID	Problema	Dependencia	Dificultad	Prioridad	Resultado
UX-003	Corregir voucher de pago	Ninguna	Media	Media	Voucher imprime correctamente
UX-004	Unificar estados de reserva	Ninguna	Baja	Media	Estados consistentes
UX-005	Eliminar disponibilidad aleatoria	Ninguna	Baja	Media	Datos reales
UX-006	Unificar formato de precio	Ninguna	Baja	Media	Formato consistente
UX-007	Corregir barra de accion en movil	Ninguna	Baja	Media	UI funcional en movil
RET-001	Corregir auth en retained-advances	Ninguna	Baja	Media	Feature funcional

FASE 4 - OPTIMIZACION (P3)

ID	Problema	Dependencia	Dificultad	Prioridad	Resultado
UX-001	Eliminar BookingForm muerto o integrar	Ninguna	Baja	Baja	Codigo limpio
UX-008	Documentar seleccion por rango	Ninguna	Baja	Baja	UX clara
PERF-001	Agregar paginacion server-side	db.ts	Alta	Baja	Mejor rendimiento
PERF-002	Optimizar findPaymentByExternalRef	db.ts	Media	Baja	Consulta eficiente
NOTIF-001	Implementar persistencia de notificaciones	Firebase	Media	Baja	Notificaciones persistentes
NOTIF-002	Agregar confirmacion por email	Nodemailer	Media	Baja	Confirmacion de reserva por email

11. CHECKLIST FINAL DE PRODUCCION

Area	Estado	Listo para Produccion?	Observaciones
Autenticacion	MEJORABLE	NO	Funciona con Firebase. Fallback inseguro. Sin rate limiting. Credenciales hardcodeadas deben eliminarse.
Usuarios	MEJORABLE	NO	CRUD protegido. Setup endpoint debe eliminarse. Registro sin confirmacion por email.
Reservas	MEJORABLE	NO	Creacion correcta con transaccion. Edicion sin transaccion. Expiracion lazy. Timezone incorrecto.
Pagos	CRITICO	NO	Sin validacion de monto en servidor. Sin idempotencia en webhooks. Status manipulable desde el cliente.
Firebase/DB	MEJORABLE	NO	Reglas de seguridad no verificadas en produccion. Varios endpoints sin proteccion. Dual write sin transaccion.
Seguridad	CRITICO	NO	6+ endpoints sin auth. Credenciales expuestas. SSRF. Setup con secreto debil. Falta rate limiting.
Administrador	CRITICO	NO	Stats y expenses publicos. Auth rota. Endpoints sin rol. Retained-advances no funcional.
UX/UI	MEJORABLE	NO	Voucher roto. Estados inconsistentes. Formulario muerto. Barra de accion oculta en movil.
Movil	MEJORABLE	NO	Buena base con bottom nav. Barra de accion oculta. Touch targets pequenos en algunos botones.
Rendimiento	MEJORABLE	NO	Sin paginacion server-side. Consultas sin limite. findPaymentByExternalRef ineficiente.
Reportes	MEJORABLE	NO	Stats basicos funcionan. Faltan reportes detallados por cancha, deporte, periodo, usuario.
Notificaciones	ERROR	NO	Sin persistencia. Sin email/SMS. Sin recordatorios. Solo toasts en memoria volatil.

APROBADA PARA PRODUCCION: NO

La aplicacion NO esta lista para produccion. Deben solucionarse obligatoriamente los 14 problemas P0 (Criticos) antes de su publicacion. Los problemas mas urgentes que requieren atencion inmediata son: (1) Proteger todos los endpoints de API con autenticacion adecuada. (2) Eliminar credenciales hardcodeadas y endpoints de setup. (3) Implementar validacion server-side de montos de pago. (4) Agregar idempotencia al webhook de Culqi. (5) Corregir la gestion de zonas horarias. (6) Asegurar que las reglas de seguridad de Firestore esten correctamente desplegadas. Una vez resueltos estos problemas criticos, la aplicacion tendria una base solida para una version beta controlada con usuarios limitados, mientras se completan los problemas P1 en la siguiente iteracion.